CSC 7073 – Principles and Practices of Machine Learning

Assignment 2 – Computer Vision: DL-based 2D Object Detection

1. Introduction and Common Setup

The objective of this assignment is to investigate the performance of a two-dimensional (2D) object detector using the well-established Ultralytics YOLO architecture. The YOLO (You Only Look Once) model operates on a grid-based intuition, processing the image with a single Convolutional Neural Network (CNN) pass and overlaying a grid where each cell is responsible for detecting objects whose centres fall inside it. To simplify training and improve bounding box predictions, the network utilises class-agnostic "anchor boxes", pre-defined reference shapes, such as wide anchors that frequently match cars and tall anchors that frequently match pedestrians.

Specifically, the pre-trained YOLOv11n model (initialised with COCO weights) is evaluated and subsequently fine-tuned on the kitti3k_yolo dataset. This dataset consists of onboard vehicle-camera images and contains bounding box annotations for three specific road-user classes: Pedestrian, Cyclist, and Car. The data is pre-divided into the following splits to facilitate training and evaluation:

- **Training Set:** 2,100 images
- **Validation Set:** 450 images
- **Test Set:** 450 images

To ensure fair and consistent comparisons across all tasks in this report, the following standardises inference settings are applied globally during evaluation (unless explicitly states otherwise):

- **Input Image Size:** 640 x 640
- **Non-Maximum Suppression (NMS) IoU Threshold:** 0.7
- **Confidence Threshold:** 0.001

2. Task 2.1 - Pre-trained YOLO Evaluation on the Car Class

2.1 Evaluation Methodology

To establish a baseline for 2D object detection, I evaluated the Ultralytics YOLO architecture (yolo11n.pt), which is pre-trained on the broad COCO dataset, prior to any fine-tuning. The evaluation was restricted strictly to the Car class (index 2) and was run on the test split of the kitti2k_yolo dataset, which contains 450 images.

2.2 Quantitative Results

To make sense of these quantitative results, it is important to define the underlying metrics:

- **Precision** evaluates the number of actual positives correctly predicted as positives divided by the number of all predicted positives (i.e. of all the bounding boxes the model predicted as cars, what fraction were truly cars).
- **Recall** evaluates the number of actual positives correctly predicted as positives divided by the number of actual positives (i.e. the proportion of all ground-truth cars in the dataset that were successfully detected by the model).

The pre-trained model evaluated 1,795 Car instances across the 450 test images. The performance metrics obtained are summarised in the table below:

Table 1: Baseline Evaluation for the Pre-trained COCO YOLO11n Model on the Car Class

Metric	Value
Car AP_{50}	0.7133
Car $AP_{50:95}$	0.4047
Precision (Car)	0.784
Recall (Car)	0.625

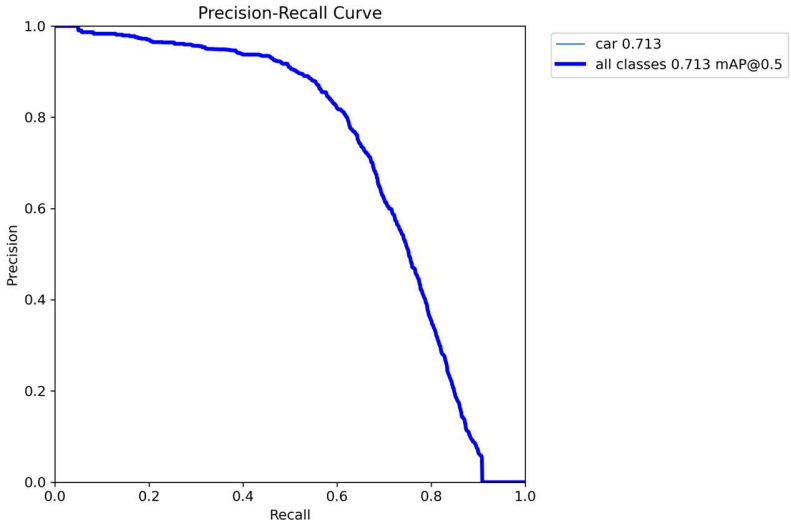


Figure 1. Precision-recall curve for the Car class, pre-trained (COCO) YOLO11n on the kitti3k_yolo test set.

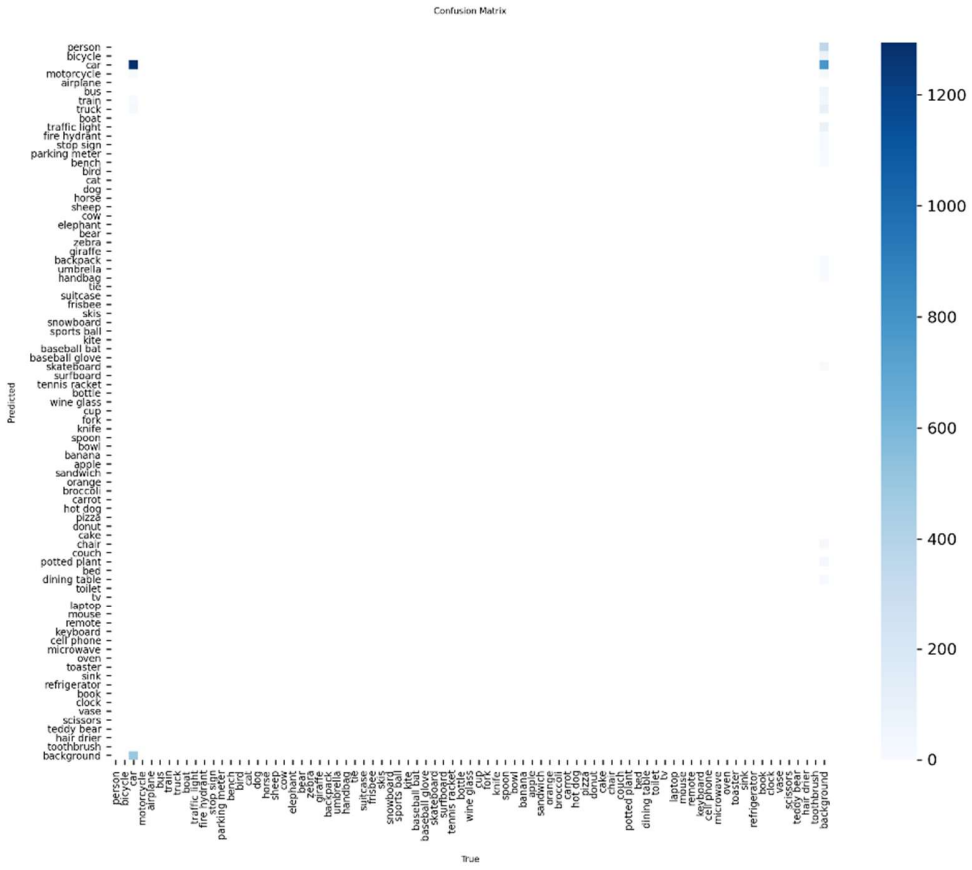


Figure 2. Multi-Class Confusion Matrix for the Pre-Trained COCO YOLO11n Model on the KITTI Test Split

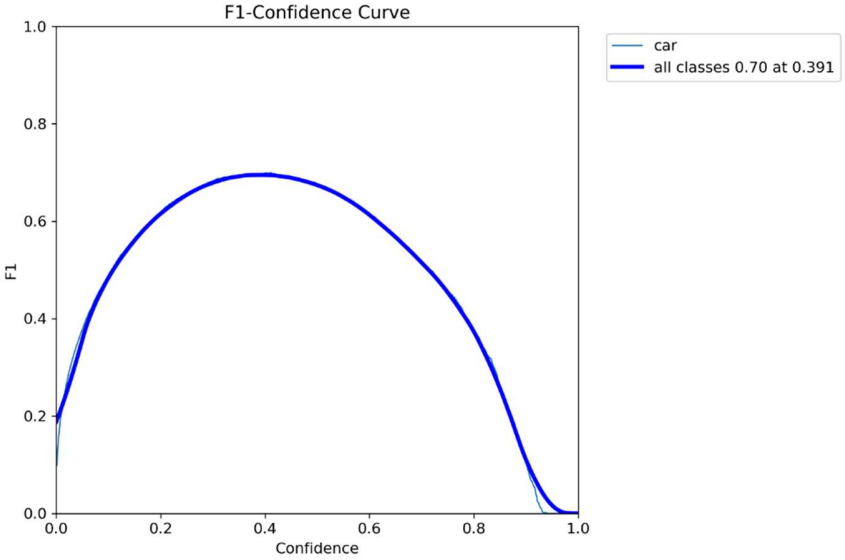


Figure 3. F1-Confidence Threshold Curve for the Pre-Trained YOLO11n Model



Figure 4. Qualitative Sample Detections and Bounding Box Predictions from the Pre-Trained Baseline

2.3 Discussion: The COCO vs. KITTI Domain Gap

The obtained performance shows a moderate AP_{50} score of 0.7133, demonstrating that the pre-trained model has a reasonable foundational ability to recognise vehicles. However, the significantly lower $AP_{50:95}$ score (0.4047) reveals a lack of bounding box precision.

Intersection over Union (IoU) measures how much the predicted bounding box overlaps with the ground truth box, meaning a higher IoU indicated better and tighter localisation. While the model succeeds at a lenient, moderate overlap threshold (AP_{50}), it struggles to tightly bind vehicles across the stricter IoU thresholds evaluated in $AP_{50:95}$ (which calculates the mean of average precision for IoU thresholds stepping from 0.50 to 0.95).

This limitation occurs because a model pre-trained on COCO does not transfer perfectly to the kitti3k_yolo dataset due to a distinct “domain gap”.

- **Camera Perspective:** The COCO dataset consists of generic web images where objects are photographed from a wide variety of random angles and distances. In contrast, the kitti3k_yolo dataset is captured specifically from an onboard vehicle camera. This means cars are almost exclusively viewed from a fixed dashcam perspective.
- **Scale and Occlusion:** In autonomous driving environments, vehicles are frequently occluded by other traffic or appear extremely small as they drive further down the road. COCO’s training data does not heavily emphasise these complex, driving-specific spatial challenges.
- **Contextual Bias:** Because YOLO learns spatial relationships and contexts, the generic backgrounds in COCO do not prepare the model for the specific urban and motorway environments found in KITTI. Fine-tuning is structurally necessary to adapt the network’s weights to the specific visual characteristics of a forward-facing road camera.

3. Task 2.2 - Fine-tuning and Hyper-parameter Comparison

3.1 Methodology and Training Configurations

To optimise the pre-trained YOLOv11n model for the KITTI-based dataset, I evaluated three distinct training configurations. These configurations employ **transfer learning**, which involves taking a neural network that has already been trained on a large dataset and adapting it to a new, usually smaller, dataset. Instead of training a detector from scratch, with random weights and requiring longer training, transfer learning utilises pre-trained weights followed by short fine-tuning on a smaller dataset. This approach is effective because the early and intermediate layers of the network have already learned to recognise visual patterns that are useful across many vision tasks.

The experiments systematically altered the frozen layers, training duration, and learning rate to contrast two common transfer learning approaches:

- **Approach 1: Feature Extraction:** The pre-trained CNN backbone is frozen, and only the new classifications head is updated.
- **Approach 2: Fine-Tuning:** The pre-trained CNN backbone is partially or fully trainable alongside the head, allowing the foundational weights to update

All configurations used a batch size of 16, an image size of 640, and the AdamW optimiser.

Table 2: Experimental Hyperparameter Settings and Transfer Learning Configurations for Fine-Tuning YOLO11n

Configuration	Epochs	Image size	Batch	lr0	Freeze	Notes
Config 1 (brief default)	20	640	16	0.001	10 (backbone frozen)	Baseline -- matches the settings given in the assignment brief
Config 2	20	640	16	0.001	0 (full fine-tune)	Tests whether unfreezing the backbone helps given only 2,100 training images
Config 3 (best)	30	640	16	0.005	0 (full fine-tune)	Longer training + higher initial LR on top of full fine-tuning

3.2 Validation results (AP_{50} / $AP_{50:95}$, all classes)

Table 3: Class-Specific and Mean Average Precision (mAP) Validation Set Results Across Fine-Tuning Configurations

Config	Pedestrian AP_{50}	Pedestrian $AP_{50:95}$	Cyclist AP_{50}	Cyclist $AP_{50:95}$	Car AP_{50}	Car $AP_{50:95}$	mAP_{50}	$mAP_{50:95}$
1 (freeze=10, 20ep)	0.6639	0.3476	0.5372	0.2985	0.8934	0.6343	0.6982	0.4268
2 (freeze=0, 20ep)	0.6542	0.3551	0.5749	0.3190	0.9031	0.6563	0.7107	0.4435
3 (freeze=0, 30ep, lr0=0.005)	0.6704	0.3651	0.6263	0.3525	0.9109	0.6729	0.7359	0.4635

3.2.1 Analysis of Tuning Strategies:

- **Impact of Unfreezing:** Comparing Configuration 1 and 2 shows that unfreezing the entire backbone yields a noticeable improvement in overall mAP_{50} (from 0.698 to 0.711). To assess multi-class performance, mean Average Precision (mAP) is used, where mAP_{50} calculates the average of AP_{50}over multiple classes, and $mAP_{50:95}$ calculates the mean of $AP_{50:95}$ over multiple classes. This metric increase indicates that the base YOLOv11 weights benefited from adapting their lower-level feature extraction specifically to the domain of autonomous driving imagery.

- **Impact of Epochs and Learning Rate:** Configuration 3 yielded the best result across all metrics. The combination of a higher initial learning rate (0.005) and a longer training schedule (30 epochs) allowed the model to navigate the loss landscape more effectively and achieve better convergence.

3.3 Final Test Set Evaluation

Given its superior validation performance, I selected Configuration 3 as the final model. I evaluated it against the unseen Test set to measure its true generalisation capabilities in real-world driving scenarios.

Table 4: Class-Specific and Overall Test Set Evaluation Metrics for the Optimal Model (Config 3)

Class	AP_{50}	$AP_{50:95}$
Cyclist	0.5347	0.3125
Pedestrian	0.6403	0.3555
Car	0.9103	0.6781
All (mAP)	0.6951	0.4487

3.3.1 Class-Specific Test Performance

The model's performance varies dramatically across different object categories, reflecting the intrinsic challenges of the dataset:

- **Car (AP_{50}: 0.9103):** The model performs exceptionally well at detecting cars. This high precision and recall can be directly attributed to the dataset's class distribution; cars make up the vast majority of the test instances (1,795 objects). This abundance provides the network with ample, varies examples to learn highly robust and generalised spatial representations.

- **Pedestrian (AP_{50}: 0.6403) & Cyclist (AP_{50}: 0.5347):** Performance drops significantly for Vulnerable Road Users (VRUs). This degradation is driven by three compounding factors:

    1. **Class Imbalance:** There is a severe lack of training examples compared to cars. In the test set, there are only 287 pedestrians and 89 cyclists.

    2. **Scale and Resolution:** Pedestrians and cyclists occupy a much smaller physical footprint within the 640x640 image frame, giving the convolutional layers fewer pixels from which to extract meaningful features.

    3. **Aspect Ratio Variability:** Unlike the rigid, generally horizontal structure of a car, pedestrians and cyclists possess highly variable, vertical aspect ratios that change drastically depending on their pose, viewing angle, and potential occlusions.

3.3.2 Overall Test Metrics

Test mAP_{50} (0.6951): At a standard intersection over Union (IoU) threshold of 50%, the model demonstrates strong overall localisation and classification.

Test $mAP_{50:95}$ (0.4487): Averaged across stricter IoU thresholds, this lower score highlights the inherent difficulty of achieving pixel-perfect bounding box alignment, a common challenge for complex scenes.

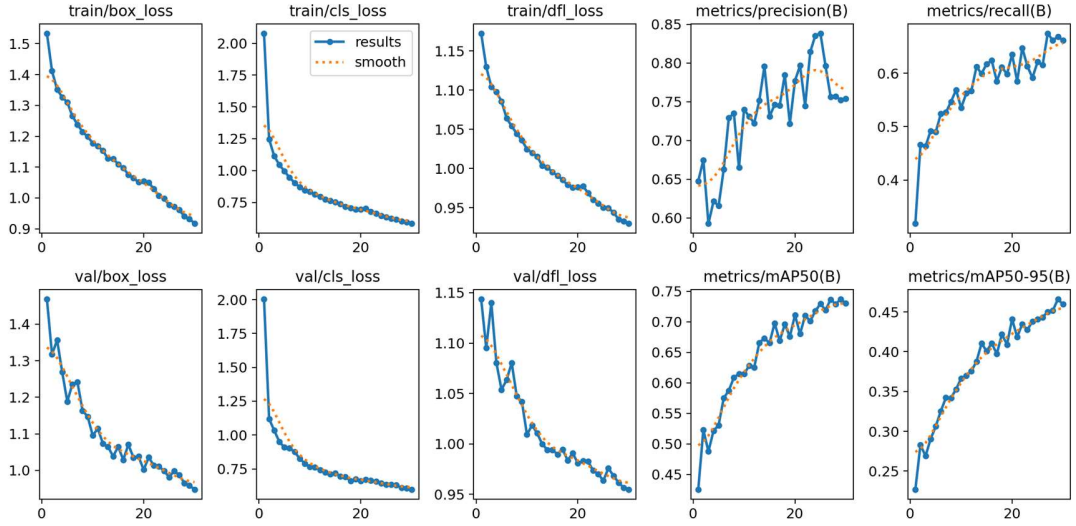


Figure 5. Configuration 3 training convergence curves (box/cls/dfl loss and mAP_{50} / $mAP_{50:95}$ over 30 epochs).

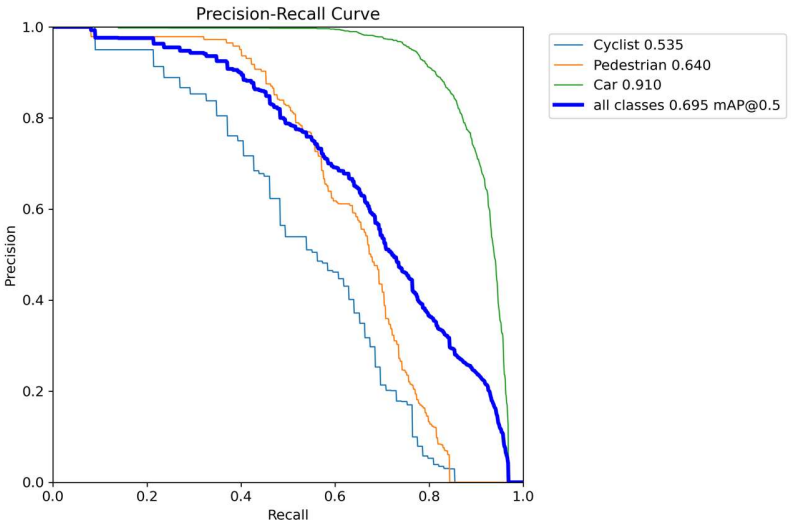


Figure 6. Configuration 3 precision-recall curve on the test set (all classes).

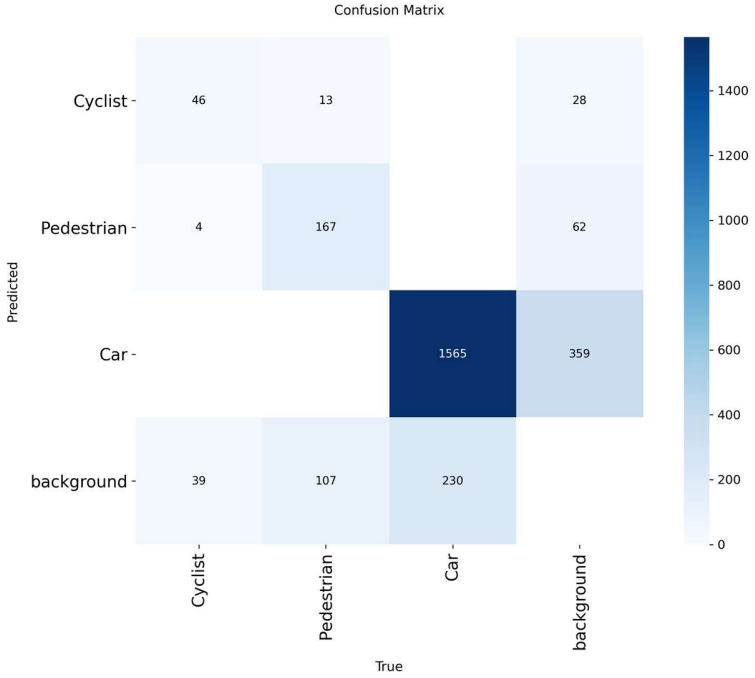


Figure 7. Configuration 3 confusion matrix on the test set.

3.3.3 Visual Diagnostics and Model Behaviour

The visual diagnostics generated during Configuration 3's evaluation corroborate these quantitative metrics and offer deeper insights into the model's learning dynamics:

- **Training Convergence Curves (Figure 5):** The loss curves illustrate stable, consistent learning over the 30 epochs. The steady decline in bounding box and classification losses indicates that the extended training duration and higher learning rate successfully navigated the loss landscape without causing severe overfitting.

- **Precision-Recall Curve (Figure 6):** The PR curve visually highlights the performance gap between classes. While the "Car" curve remains high and stable across recall levels, the "Pedestrian" and "Cyclist" curves experience a steeper drop-off, demonstrating the model's struggle to maintain precision when trying to retrieve all minority class instances.

- **Confusion Matrix (Figure 7):** The confusion matrix pinpoints the exact nature of the model's errors. It reveals the network's ongoing struggle with the minority classes, confirming that while cars are easily distinguished, the model still faces challenges isolating smaller, underrepresented objects from the background.

3.4 Pre-trained vs. Fine-Tuned – Car Class

Table 5: Performance Comparison on the Car Category: Pre-Tuned COCO Baseline vs. Fine-Tuned Model (Config 3)

Model	Car AP_{50}	Car $AP_{50:95}$
Pre-trained (COCO), T2.1	0.7133	0.4047
Fine-tuned, Config 3 (test set)	0.9103	0.6781

3.4.1 Did Fine-Tuning Improve Performance?

While the baseline pre-trained model would likely detect cars reasonably well due to its prior knowledge, fine-tuning bridges the gap between generic intelligence and domain expertise. It aligns the model’s decision boundaries with the specific environmental conditions (lighting, road textures etc.) and ground-truth labelling criteria of the target dataset. This directly translates to tighter bounding box localisations, higher recall, and the exceptional 0.9103 precision achieved in the final test evaluation.

4. Task 2.3 - One-class Fine-tuning: Car-only Detector

4.1 Methodology and Experimental Setup

To evaluate the impact of class specialisation on object detection performance, YOLOv11n was fine-tuned on the single-class dataset kitti3k_car_only. This dataset shares the exact same image split (2,100 training, 450 validation, and 450 test images) as the main KITTI dataset, but all non-car annotations (Pedestrians and Cyclists) were filtered out.

I adopted the optimal configuration established in Task 2.2 for the training setup:

- **Model:** yolo11n.pt (pre-trained COCO weights)
- **Backbone Training:** Unfrozen (freeze=0)
- **Epochs:** 30
- **Batch Size:** 16
- **Image Size:** 640 x 640
- **Initial Learning Rate:** 0.005 (AdamW optimiser)

4.2 Test Set Evaluation Results

After fine-tuning on the kitti3l_car_only training set for 30 epochs, the best checkpoint was evaluated on the unseen test set (450 images, 1,795 car instances).

Table 6: Quantitative Test Set Evaluation Metrics for the Specialised Single-Class (Car-Only) Detector

Metric	Score	Interpretation
Car Precision (P)	0.8990	High proportion of predicted car bounding boxes correspond to actual cars
Car Recall (R)	0.8250	The detector successfully retrieves 82.5% of all ground-truth cars
Car AP_{50}	0.9147	Outstanding localisation precision at 50% IoU threshold
Car $AP_{50:95}$	0.6792	Robust performance across strict IoU thresholds

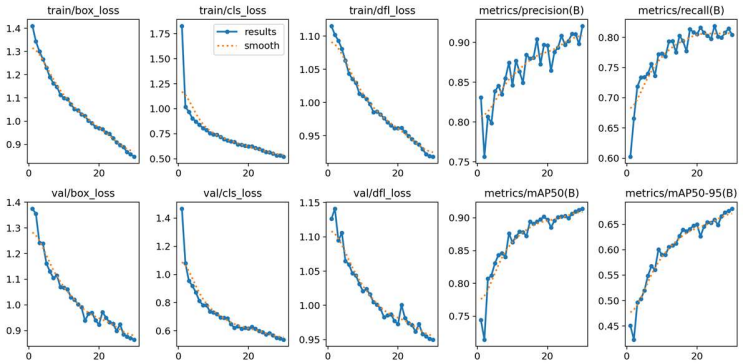


Figure 8. Training and Validation Loss Curves alongside Precision, Recall, and mAP Metrics for the Single-Class (Car-Only) Detector over 30 Epochs

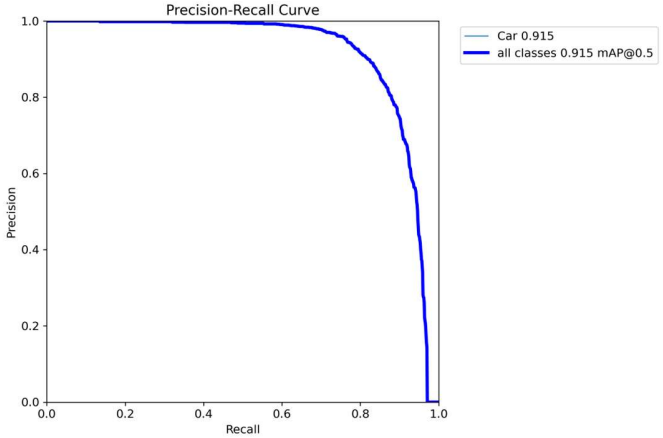


Figure 9. Precision-Recall Curve for the Specialised Single-Class (Car-Only) Model Evaluated on the Unseen Test Set

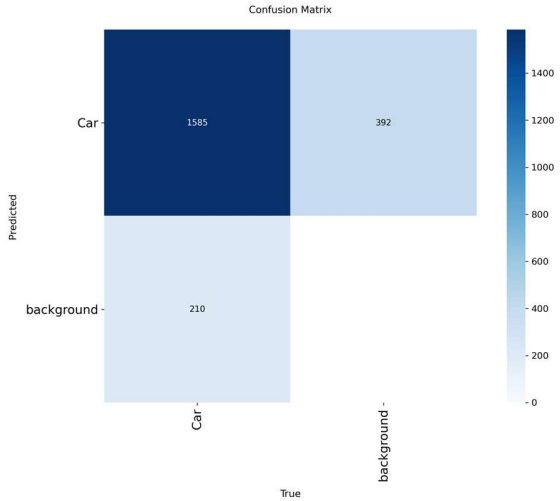


Figure 10. Car-only confusion matrix on the test set.

4.3 Performance Comparison: Single-Class vs Three-Class Model

Comparing the car detection performance of the specialised single-class model against the original multi-class model from Task 2.2 reveals a clear advantage in narrowing the task domain:

Table 7: Performance Comparison on the Car Category: Specialised Single-Class Detector vs. Multi-Class Model

Metric	One-class (Car-only) test set	Three-class model, Car (T2.2, test set)
Car AP_{50}	0.9147	0.9103
Car $AP_{50:95}$	0.6792	0.6781

4.3.1 Why Specialisation Improves Performance:

1. **Dedicated Representation Capacity:**

In a multi-class model, the detection head and feature extraction layers must share capacity to classify diverse geometries (e.g., vertical/variable pedestrian poses vs horizontal rigid car bodies). In the car-only model, 100% of the network's classification parameters are dedicated exclusively to learning car features.

2. **Elimination of Gradient Interference:**

In multi-class training, noisy gradients from minority classes (such as Pedestrians or Cyclists) can compete with car feature optimisation. Removing minority classes eliminates cross-class confusion and gradient conflicts during backpropagation.

3. **Simplified Detection Head & Multi-task Loss:**

The YOLO architecture utilises a multi-task loss function that combines three distinct components: Objectness (L_{obj}), Bounding Box (L_{box}), and Classification (L_{class}). This is mathematically represented as:

$$L = L_{obj} + y_{obj}(\lambda_{box}L_{box} + \lambda_{class}L_{class})$$

By reducing the number of classes to one, the network entirely removes the need to calculate the categorical cross-entropy loss for classification (L_{class}). The network only needs to resolve Objectness and bounding box localisation without allocating loss penalties to inter-class misclassifications (e.g. confusing a cyclist with a car or pedestrian). This directly translates into tighter bounding box regression, explaining the increases in AP_{50} and $AP_{50:95}$.

5. Task 2.4 - Qualitative Analysis of Detection Results

5.1 Experimental Setup and Threshold Definitions

To qualitatively assess the YOLOv11 models, I visualised predictions from Configuration 1 (baseline fine-tune) and Configuration 3 (best fine-tune) on three validation images: 000136, 000232, and 000189. I plotted ground truth (GT) bounding boxes in blue (dashed).

During inference, it is common for many grid cells to detect the same object. To resolve this, the network relies on Non-Maximum Suppression (NMS) to convert noisy raw predictions into a clean final set of detections. NMS achieves this by selecting the highest-scoring class-specific prediction and removing all predicted boxes that have a significant overlap with it.

Model predictions were matched to GT boxes using the following criteria:

- **Confidence Threshold (CONF_THRESH):** Set to 0.50. The model only plots boxes for objects it is at least 50% confident about.
- **IoU Matching Threshold (MATCH_IOU_THRESH):** Set to 0.50. A predicted box must overlap a ground truth box by at least 50% to be considered a correct match.

Based on this matching, the boxes were colour-coded:

- **Green (True Positive – TP):** Correctly predicted bounding boxes. Formally, a predicted box is a TP if it has the correct class, its IoU with a ground-truth box is above the threshold, and that ground-truth box has not already been matched to a higher-confidence prediction.
- **Red (False Positive – FP):** Formally, a predicted box is an FP if its IoU with every eligible ground-truth box is below the IoU threshold, if it is a duplicate prediction for an object already matched, or if its predicted class is incorrect.
- **Amber (False Negative – FN):** Formally, a ground-truth box is considered an FN if no prediction matched it.



Figure 11. Qualitative comparison across the three validation images (rows) for Configuration 1 (left column) and Configuration 3 (right column). Blue = ground truth, green = TP, red = FP, amber = FN.

5.2 Impact of Changing Thresholds

Adjusting the confidence and IoU thresholds directly alters the balance between True Positive, False Positives, and False Negatives:

- **Varying Confidence Threshold:** As a general rule, a lower confidence threshold means more boxes are retained, which usually increases recall but can cause precision to decrease. If we *increase* the confidence threshold (e.g. to 0.80), the model becomes more conservative. It will make fewer predictions, which generally reduces False Positives (Red) but increases False Negatives (Amber) because it suppresses less certain, yet potentially correct, detections. *Decreasing* the threshold (e.g. to 0.20) will result in more predictions, decreasing False Negatives but increasing False Positives.
- **Varying IoU Threshold:** If we *increase* the IoU threshold (e.g. to 0.75), the criteria for a match becomes much stricter. Bounding boxes must align almost perfectly with the ground truth. This will convert some True Positives (Green) into False Positives (Red, because the prediction isn't tight enough) and False Negatives (Amber, because the GT is considered undetected). *Decreasing* the IoU threshold accepts looser bounding boxes as True Positives.

3. **Comparison: Configuration 1 vs. Configuration 3**

By observing the qualitative comparisons in *Figure 8:*

- **Configuration 1 (Baseline):** The baseline model typically shows a higher presence of Amber (FN) and Red (FP) boxes. Because its lower-level features were frozen and it was trained for fewer epochs at a lower learning rate, it struggles to detect smaller or partially occluded objects,

leading to more False Negatives (missed pedestrians/cyclists or distant cars). It may also struggle with tight localisation, occasionally returning False Positives.

- **Configuration 3 (Best):** The full fine-tuned model demonstrates visibly superior detection quality. There is a noticeable increase in Green (TP) boxes and a reduction in Amber (FN) boxes, indicating higher recall. The bounding boxes are generally tighter and better aligned with the blue dashed GT boxes. The model is distinctly better at identifying minority classes and smaller objects in the background without introducing excessive False Positives (Red).

4. Potential Impact of False Positives and False Negatives in Autonomous Driving

In the context of a camera-based 2D object detection system for autonomous vehicles, both errors carry significant risks:

- **False Negatives (Amber):** This is the most critical safety hazard. If a system fails to detect a pedestrian stepping into the road or a car braking ahead, the autonomous vehicle will not engage emergency protocols, likely resulting in a severe or fatal collision. High recall (minimising FNs) is paramount for safety.
- **False Positives (Red):** While generally less deadly than FNs, FPs are highly disruptive. If the system detects a phantom pedestrian or car that isn't actually there, the vehicle may execute an emergency stop on a clear road. This can cause rear-end collisions from following vehicles, disrupt traffic flow, and damage passenger trust and comfort.

6. Task 2.5 - Interpreting YOLO Using EigenCAM

6.1 Methodology and Setup

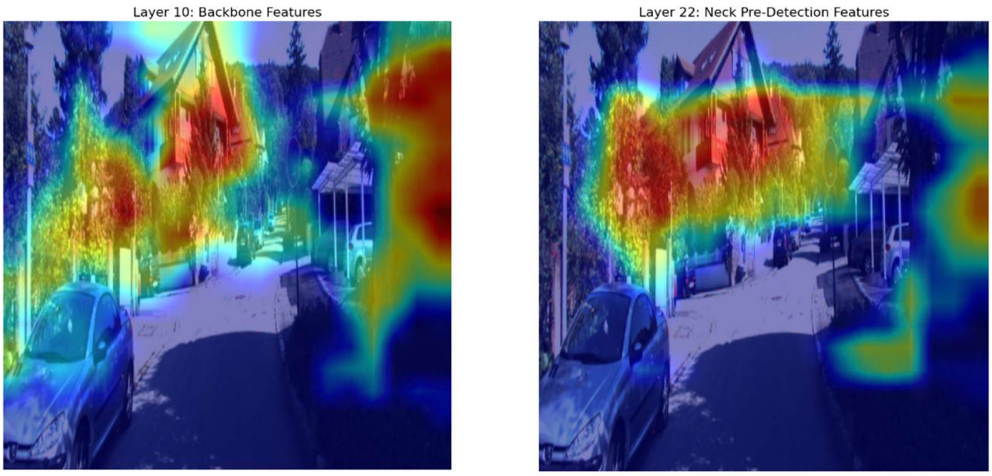
To interpret the internal representations learned by the fine-tuned YOLO model, I used the EigenCAM library to generate visual heatmaps. These heatmaps highlight the specific image regions that most strongly influence the network's predictions.

I performed the evaluation using the best-performing model checkpoint from Task 2.2 on a sample validation image (000136). Because the YOLO architecture outputs a tuple, I implemented a custom PyTorch wrapper (YOLOWrapper) to extract the main prediction tensor, ensuring compatibility with the EigenCAM processing pipeline. I then overlaid the resulting greyscale heatmaps onto the original RGB image for qualitative assessment.

6.2 Target Layer Selection and Justification

I selected two distinct layers of the YOLO architecture to observe the progression of feature learning from the bottom to the top of the network. This aligns with the core theoretical principle that a CNN learns visual features progressively through repeated convolution, non-linear activation (e.g. ReLU), and pooling blocks; earlier layers detect simple patterns, whereas deeper layers detect more complex patterns and semantic concepts.

- **Layer 10 (End of the Backbone):** The YOLO architecture utilises a fully convolutional design where the image is processed once by a shared convolutional backbone. This layer captures deep semantic features. Because of stacked convolutions and pooling, every grid cell at this stage contains information about objects extending beyond that cell's physical dimensions, allowing the network to capture broad scene context.
- **Layer 22 (End of the Neck):** This layer contains pre-detection localised features. It sits just before the final detection heads; I selected it to demonstrate how the model aggregates and refines spatial data to confidently isolate specific objects.



Figures 12 & 13. EigenCAM heatmap at Layer 10 (end of backbone) and Layer 22 (end of neck), image 000136. Demonstrating Global Scene Context Representations and Spatially Localised Pre-Detection Target Features respectively.

6.3 Heatmap Analysis and Discussion

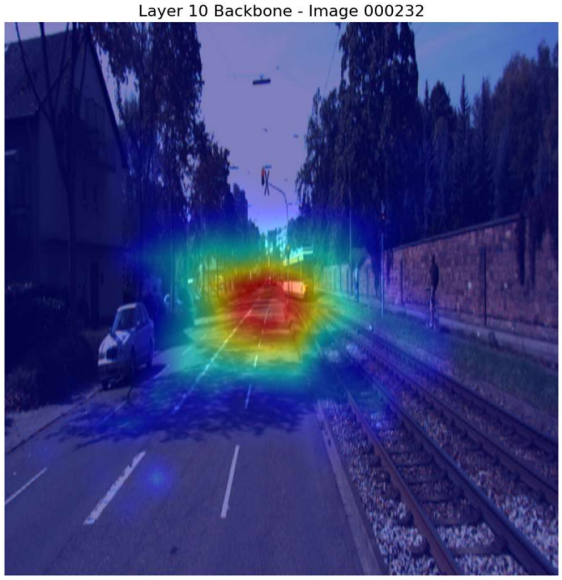
Analysing the generated visualisations reveals key insights into the detector's behaviour:

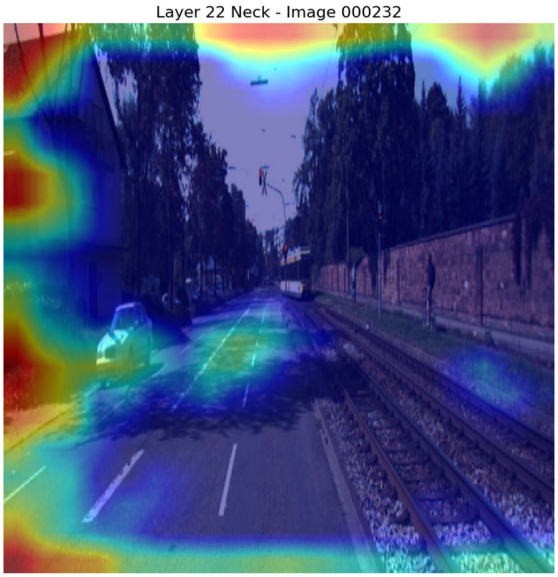
- **Distinct Layer Behaviours:** The two layers behave remarkably differently, illustrating the hierarchical nature of deep convolutional networks. The heatmap for Layer 10 shows broad, generalised activations across the scene. It highlights contextual structures, road boundaries, and general shapes, indicating that the backbone is focused on gathering rich, global image features. Conversely, the heatmap for Layer 22 is highly localised. The activations here are tightly constrained to specific spatial locations, representing the network's shift from understanding the 'whole scene' to pinpointing individual targets.
- **Intuitive Understanding:** The heatmaps are highly intuitive. They confirm that the model behaves rationally, progressively filtering out background noise as data passes from the backbone into the neck.
- **Explaining Successful Detections:** The layer 22 heatmap is particularly useful for explaining why the model successfully detects objects. The intense 'hot' regions on the map align directly over the vehicles and relevant objects in the image. This proves that the model's predictions are genuinely driven by the visual features of the cars themselves (e.g. chassis, wheels, windshields etc.), rather than relying on unreliable correlations, dataset biases, or irrelevant background artifacts.

7. Conclusion

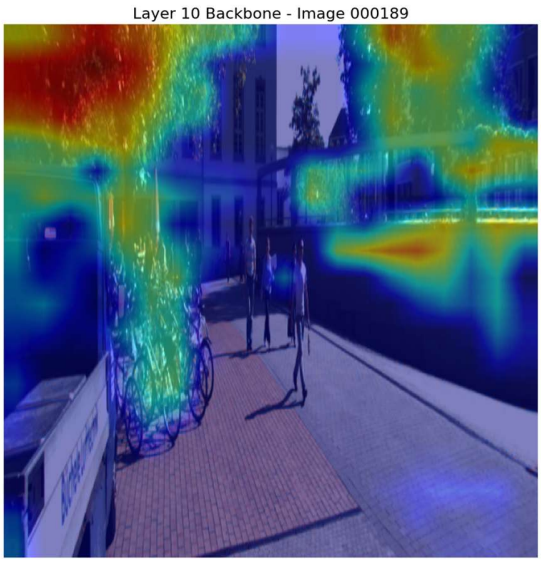
The evaluation of the YOLO architecture on the kitti3k_yolo dataset underscores the necessity of domain-specific fine-tuning for autonomous driving applications. While the COCO-pre-trained baseline provided a solid foundation, fine-tuning significantly improved mAP metrics across all classes by adapting the network to the specific perspectives and object scales of onboard vehicle cameras. Furthermore, task simplification via single-class fine-tuning (Car-only) yielded the tightest bounding box localisations by eliminating inter-class gradient interference. Finally, the EigenCAM visual analysis confirmed that the network's predictive power is grounded in extracting genuine object features rather than relying on background artifacts or correlations, demonstrating that the fine-tuned model is both highly accurate and visually robust.

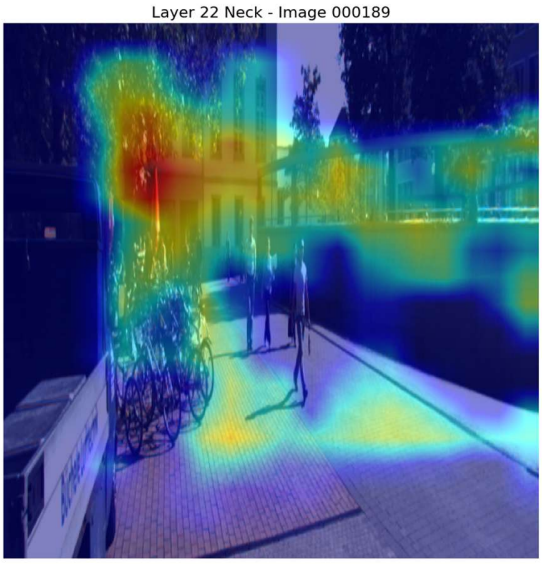
8. Appendix





Figures 14 & 15 EigenCAM heatmap at Layer 10 (end of backbone) and Layer 22 (end of neck), image 000232.





Figures 16 & 17EigenCAM heatmap at Layer 10 (end of backbone) and Layer 22 (end of neck), image 000189.